
DDLJ Trading System v9.1.0

Strategy Core Documentation

A Lego-Piece Guide to Every Module in the Engine

Version 9.1.0 | Release Date: April 2026

Platform: Zerodha Kite | Instruments: BankNifty, Nifty Options

Timeframes: 15-minute (entry) + 60-minute (bias)

This document is your complete, self-contained reference for every module inside the DDLJ Trading System. Each module is explained as a separate 'lego piece' — a self-contained block with a clear input, a clear output, and a single job. When you snap all the pieces together, you get the full trading engine. The language is kept deliberately simple so that anyone — even someone brand new to trading or programming — can follow along. Think of it like assembling a model from a kit: every piece has a picture and a purpose, and the instructions show you exactly where each piece goes.

Throughout this guide you will find tables, analogies, and step-by-step logic chains. If a section feels too technical, skip ahead to the analogy call-out and come back to the details later. The goal is understanding, not memorization. Welcome to the engine room.

Table of Contents

Table of Contents	2
1. The Big Picture	5
1.1 The Pipeline	5
1.2 Two Timeframes	6
2. Module 1 — Config (The Control Panel)	7
2.1 Key Config Categories	7
2.2 Runtime Overrides	8
3. Module 2 — Indicators (The Toolbox)	9
3.1 Indicator Reference	9
3.2 Why These Indicators Matter for Trend Following	9
4. Module 3 — Bias Engine (The Compass)	11
4.1 Three Independent Bias Signals	11
4.2 Decision Priority System	11
4.3 Safety Check: ATR Overextension	12
5. Module 4 — Signal Engine (The Trigger)	13
5.1 Two Entry Patterns	13
5.2 Signal Filters	13
5.3 Stop Loss and Target Calculation	14
6. Module 5 — Options Engine (The Translator)	15
6.1 Strike Selection	15

6.2 IV Estimation from VIX	15
6.3 Spread Modeling (VIX Regimes)	15
6.4 Position Sizing Formula	16
7. Module 6 — Black-Scholes (The Calculator)	17
7.1 The Greeks — Explained Simply	17
7.2 Why Real VIX Data Matters	17
8. Module 7 — Cost Calculator (The Accountant)	19
8.1 Zerodha Transaction Costs Breakdown	19
8.2 Cost Impact on Trading Decisions	19
9. Module 8 — Data Fetcher (The Pipeline)	20
9.1 Auto-Chunking and Caching	20
9.2 Rate Limiting and Retry Logic	20
10. Module 9 — Token Manager (The Gatekeeper)	22
10.1 Token Lifecycle	22
10.2 Daily Token Refresh	22
11. Complete Strategy Flow	23
11.1 Step-by-Step Walkthrough	23
11.2 Position Management Rules	24
11.3 Exit Handling Process	24
12. Lego Piece Connections	25
12.1 Data Flow Diagram (Textual)	25
12.2 Module Dependency Map	25

13. Key Design Decisions	27
13.1 Why Two Timeframes?	27
13.2 Why Three-Signal Bias?	27
13.3 Why ATR-Based Stops?	27
13.4 Why ITM Options?	28
13.5 Why Real VIX Data?	28

1. The Big Picture

The DDLJ Trading System is an EMA-based trend-following options trading engine built for the Indian markets. It operates exclusively on BankNifty and Nifty index options through the Zerodha Kite platform. The name DDLJ is a nod to the iconic Bollywood film — because just like the movie's famous 'palat' moment (turn around!), the system is always watching for the market to turn. But unlike a movie, the system does not rely on emotions. It relies on math, structure, and a crystal-clear pipeline of decisions.

At its heart, DDLJ answers three questions every single trading day: (1) Which direction is the market likely to move? (2) Is this the right moment to enter a trade? (3) Which specific option contract should I buy or sell to express that view? Each question is handled by a dedicated module — a 'lego piece' — and the modules snap together in a fixed order to produce a final trading decision. No module talks to another module out of turn. Information flows in one direction, like water through a pipe, and each stage adds its own filter before passing the result downstream.

1.1 The Pipeline

The entire system can be visualized as a five-stage pipeline. Raw market data enters at the top, and a fully specified trade order comes out at the bottom. Every stage is independent, testable, and replaceable. If you want to change how bias is calculated, you only touch the Bias Engine — nothing else needs to change. This is the power of the lego-piece architecture.

Stage	Module Name	Input	Output	Analogy
1	Data Fetcher	Kite API credentials	OHLCV candles	The radar dish collecting signals
2	Bias Engine	60m candles + indicators	BULLISH / BEARISH / NEUTRAL	Intelligence: 'Enemy is north'
3	Signal Engine	15m candles + bias	Entry signal + SL/Target	Timing: 'Strike now!'
4	Options Engine	Signal + spot price + VIX	Option order (strike, type, qty)	Weapon selection
5	Trade Recorder	Executed order	P&L; and journal entry	After-action report

The military analogy is intentional and helpful. In a military operation, you would never fire a weapon without intelligence telling you where the target is (Bias), and you would never strike without a clear go-order (Signal). The DDLJ system follows the same discipline. Bias sets the direction. Signal sets the timing. Options set the instrument. Each layer adds confidence and specificity before money is put at risk.

1.2 Two Timeframes

DDLJ uses two timeframes, and understanding why is critical to understanding the entire system. The 60-minute timeframe is the 'big picture' — it tells you the overall trend. Think of it like looking at a map from an airplane: you can see which way the river flows, but you cannot see individual rocks. The 15-minute timeframe is the 'close-up' — it tells you the precise moment to enter. Think of it like a magnifying glass on the map: now you can see the rock you need to step on. Using only one timeframe would be like trying to navigate with either only the map or only the magnifying glass — incomplete and dangerous.

The 60m timeframe feeds the Bias Engine exclusively. The 15m timeframe feeds the Signal Engine. The Signal Engine never determines direction — it only determines timing. Direction comes from above. This separation of concerns is what makes the system robust. Even if a 15m chart shows a brief dip, the 60m bias will keep you from taking a counter-trend trade. The big picture always wins over the close-up when there is a conflict.

2. Module 1 — Config (The Control Panel)

The Config module is the master control panel of the DDLJ system. Imagine a spacecraft cockpit with hundreds of switches and dials — each one controls a specific behavior, and every one has a sensible default so the ship flies even if you never touch a single switch. The Config module holds every adjustable parameter in one centralized location so that no magic numbers are hidden inside other modules. If a value affects how the system behaves, it lives in Config.

This design philosophy has a name: 'single source of truth.' When you want to change the risk per trade from 2% to 1.5%, you change one number in Config and every module that reads risk percentage instantly sees the new value. There are no copies, no stale values, no surprises. The Config is loaded once at startup and then shared as a read-only object to every other module. No module modifies Config at runtime — it only reads from it.

2.1 Key Config Categories

Category	Example Parameters	Purpose
Capital & Risk	initial_capital, risk_per_trade_pct, max_daily_loss	How much money to deploy and how much to risk
Market Timing	market_open, market_close, force_close_time	When the system is allowed to trade
Instruments	primary_index, exchange, lot_size	Which index and exchange to trade on
Bias Engine	bias_ema_period, bias_atr_multiplier, structure_lookback	How the compass determines direction
Signal Engine	signal_ema_fast, signal_ema_slow, min_rr_ratio	How the trigger decides timing
Position Mgmt	trail_after_pct, breakeven_after_pct, near_target_pct	How open positions are managed after entry
Options	strike_selection_mode, iv_fallback_pct, spread_regime	How spot signals become option orders
Position Sizing	sizing_method, max_lots, min_lots	How many lots to trade per signal
Timeframes	bias_tf, signal_tf, candle_interval	Which timeframes feed which engine
Paper Trading	paper_mode, log_trades, simulate_slippage	Safe mode for testing without real money

2.2 Runtime Overrides

While Config is read-only during normal operation, the system supports runtime overrides for specific scenarios. For example, on high-volatility event days (like RBI policy announcements or election results), a trader may want to reduce position size or widen stop losses without editing the config file and restarting the system. The override mechanism works like a transparent layer: the original config value remains unchanged, but the override value takes precedence as long as it is active. When the override is cleared, the system seamlessly falls back to the original value. This ensures that temporary adjustments never permanently alter the system's baseline behavior. All overrides are logged so there is a complete audit trail of every parameter change during a trading session.

Runtime overrides are especially useful during paper-trading phases. A developer can test aggressive configurations during market hours and then revert to conservative defaults with a single command. The override system also supports scheduled overrides — for instance, automatically reducing lot size during the last 30 minutes of the trading day when volatility tends to spike. This kind of dynamic risk management would be impossible if every parameter were hard-coded into the modules.

3. Module 2 — Indicators (The Toolbox)

The Indicators module is a collection of pure mathematical functions — a toolbox of instruments that other modules reach for when they need to measure something about the market. These functions have no memory and no opinions. You hand them raw numbers, and they hand you back a result. They never decide whether to trade or not — that is the job of the Bias and Signal engines. The toolbox simply provides the measurements that those engines use to make their decisions.

Keeping indicators in their own module is a deliberate design choice. It means you can test any indicator independently by feeding it known inputs and checking the output — like calibrating a thermometer with boiling water and ice water. If the indicator passes its unit tests, you can trust it inside the trading engine. If you ever want to replace EMA with a different smoothing algorithm, you swap it out in this one module and every downstream consumer automatically gets the new calculation.

3.1 Indicator Reference

Indicator	Input	Output	Kid-Friendly Explanation
EMA (Exponential Moving Average)	Price series + period N	Single smoothed value per bar	A smoothed average that reacts faster to recent moves — like a line that hugs the price but ignores small wiggles
ATR (Average True Range)	High, Low, Close series + period N	Single volatility value per bar	How much does the market typically move in one period? Like measuring the average height of ocean waves
Slope	A numeric series + lookback window	Rate of change per bar	Is the line going up, going down, or going flat? Like checking whether a hill is uphill, downhill, or level
Swing High	High prices + lookback N	Price level of the last peak	The top of the most recent hill — where the price turned down before
Swing Low	Low prices + lookback N	Price level of the last valley	The bottom of the most recent valley — where the price turned up before
VWAP (Volume Weighted Average Price)	Price + Volume series	Average price weighted by volume	The 'fair price' that most traders actually paid — like the average bill at a restaurant

3.2 Why These Indicators Matter for Trend Following

Trend-following systems live and die by their ability to (a) detect that a trend exists, and (b) measure how strong it is. EMA provides the trend detection — when price is above the EMA, the trend is up; when below, the trend is down. ATR provides the strength measurement — a wide ATR means the market is moving aggressively, while a narrow ATR means the market is quiet and the trend may be fading. Slope adds a third dimension: not just 'is price above or below the EMA' but 'is the EMA itself rising or falling?' A rising EMA with a positive slope is much stronger confirmation than a price that is barely above a flat EMA.

Swing highs and lows are the system's reference points for setting targets and stop losses. When you enter a bullish trade, the most recent swing high is a natural target — that is where sellers previously stepped in. When you enter a bearish trade, the most recent swing low is the natural target — that is where buyers previously supported the price. Using structure-based targets instead of fixed-percentage targets makes the system adaptive. In a wide-ranging market, targets are wider; in a narrow market, targets are tighter. This adaptiveness is one of the key advantages of a structure-aware trading system.

4. Module 3 — Bias Engine (The Compass)

The Bias Engine is the system's compass. Its sole job is to answer one question: 'Which direction is the market most likely to move in the near future?' It produces one of three outputs: BULLISH, BEARISH, or NEUTRAL. It operates exclusively on the 60-minute timeframe, which means it looks at the big picture and ignores short-term noise. A 15-minute dip does not change the bias — only a sustained move on the hourly chart can do that. This is by design: the compass is deliberately slow to change direction, because a compass that spins wildly is worse than no compass at all.

4.1 Three Independent Bias Signals

The Bias Engine does not rely on a single indicator to make its call. Instead, it consults three independent signals and weighs them together. This is like asking three different scouts for their opinion before making a military decision — if all three agree, you have high confidence; if they disagree, you proceed with caution or stay put. The three signals are: EMA Position (is price above or below the EMA?), EMA Slope (is the EMA rising or falling?), and Market Structure (are swing highs and lows forming a pattern that supports a direction?). Each signal is binary — it can only say bullish or bearish — but the combination of three signals gives you a much richer picture than any single signal alone.

Signal	Bullish Condition	Bearish Condition	Weight
EMA Position	Close > EMA(60m)	Close < EMA(60m)	Primary
EMA Slope	Slope > 0 (EMA rising)	Slope < 0 (EMA falling)	Confirming
Market Structure	Higher Highs + Higher Lows	Lower Highs + Lower Lows	Confirming

4.2 Decision Priority System

Not all signals are created equal. The EMA Position is the primary signal — it carries the most weight because it directly measures whether price is above or below the smoothed average. The other two signals (Slope and Structure) are confirming signals — they cannot override EMA Position, but they can strengthen or weaken the final bias. The decision logic works in three levels of confidence:

Level	Condition	Bias Output	Confidence
Strong	All 3 signals agree	BULLISH or BEARISH	High
Moderate	2 of 3 signals agree (must include EMA Position)	BULLISH or BEARISH	Medium
Weak / No Bias	Signals disagree or EMA Position is neutral	NEUTRAL	Low

4.3 Safety Check: ATR Overextension

Even when all three signals agree on a direction, the Bias Engine performs one final safety check before issuing its verdict. It measures how far the current price has moved away from the EMA in terms of ATR multiples. If price has moved more than 3x ATR away from the EMA, the engine considers the market overextended — like a rubber band stretched too far — and overrides the bias to NEUTRAL. This prevents the system from entering a trend at the very end of its run, when a reversal is statistically more likely than a continuation. The 3x ATR threshold is configurable via the Config module, but 3x has proven robust across backtesting. The overextension check is the system's way of saying: 'Yes, the compass says north, but we have already traveled too far north too fast — let us wait for a pullback.'

5. Module 4 — Signal Engine (The Trigger)

While the Bias Engine tells you which direction to face, the Signal Engine tells you when to pull the trigger. It operates on the 15-minute timeframe and looks for precise entry patterns that align with the current bias. A signal without a bias is like a gun without a target — dangerous and pointless. A bias without a signal is like a target without a gun — frustrating and useless. The system needs both to trade. The Signal Engine receives the bias as an input and only looks for entries in the same direction. If the bias is BULLISH, it only hunts for buy signals. If BEARISH, only sell signals. If NEUTRAL, it stays idle.

5.1 Two Entry Patterns

The Signal Engine recognizes two distinct entry patterns, each designed to capture a different type of market behavior. The EMA Cross pattern catches fresh trend reversals — when the fast EMA crosses above the slow EMA, a new uptrend may be starting. The Continuation pattern catches pullbacks within an existing trend — when price pulls back to the EMA and then bounces, the trend is resuming. Both patterns are tried-and-true methods used by professional trend followers worldwide.

Pattern	Bullish Trigger	Bearish Trigger	Best For
EMA Cross	Fast EMA crosses above Slow EMA	Fast EMA crosses below Slow EMA	Catching new trend reversals early
Continuation	Price pulls back to EMA and bounces up	Price pulls back to EMA and bounces down	Entering during a pullback in an existing trend

5.2 Signal Filters

Not every EMA cross or continuation bounce is worth trading. The Signal Engine applies a set of filters to reject low-quality signals before they ever reach the Options Engine. Think of these filters as a sieve — the coarse mesh catches the obviously bad signals, and the fine mesh catches the subtle ones. A signal must pass every single filter to be accepted. Failing even one filter means the signal is rejected.

Filter	Rule	Reason
Body Size	Candle body must be > 30% of total range	Avoid doji-like candles with no conviction
Minimum RR	Risk-to-reward ratio must be $\geq 1.5:1$	Never take a trade where potential loss exceeds potential gain by too much
Bias Alignment	Signal direction must match bias	Never trade against the 60m trend direction
ATR Minimum	ATR must be above a minimum threshold	Avoid entering in dead, flat markets with no movement
Volume Check	Volume on signal bar must be above average	Confirm the move has participation, not just a handful of traders

5.3 Stop Loss and Target Calculation

Once a signal passes all filters, the engine calculates the stop loss and target levels. The stop loss is always set at 2x ATR from the entry price. Why 2x ATR? Because 1x ATR would get stopped out by normal market noise, and 3x ATR would risk too much capital. Two times ATR gives the trade enough room to breathe while still keeping the risk manageable. The target is set at the most recent swing high (for bullish trades) or swing low (for bearish trades), as identified by the Indicators module. If the swing level does not offer a minimum reward-to-risk ratio of 1.5:1, the signal is rejected — even if it passed all other filters. This ensures that every accepted trade has a favorable mathematical expectation from the very start.

6. Module 5 — Options Engine (The Translator)

The Options Engine is the translator between the spot market and the options market. The Signal Engine produces a signal on the spot index (BankNifty or Nifty), but you cannot trade the spot index directly — you must trade options on it. The Options Engine takes the spot signal and converts it into a specific option contract with a defined strike price, type (CE/PE), quantity, and estimated cost. It is like translating a military order from 'attack the hill' into a specific plan: 'Squad Alpha moves to Grid Reference 47 at 0600 hours with 20 soldiers.' The translation must be precise, because a mis-translated order can result in buying the wrong strike, paying too much premium, or taking on unintended risk.

6.1 Strike Selection

The system supports three strike selection modes, each appropriate for different market conditions and trading styles. ATM (At-The-Money) strikes are the cheapest but have the highest gamma risk — they are very sensitive to small price moves. ITM (In-The-Money) strikes have intrinsic value and behave more like the underlying index, making them more predictable. DEEP_ITM strikes are the most expensive but move almost one-to-one with the index, offering the most delta exposure per rupee of premium. The default mode is ITM, which balances cost and responsiveness. The strike selection mode is configured in the Config module and can be overridden at runtime for special situations.

Mode	Strike Distance	Delta Approx.	Character
ATM	Closest to current spot	0.45 - 0.55	Cheap, high gamma, fast-decaying
ITM	1-2 strikes in the money	0.55 - 0.70	Balanced cost and responsiveness
DEEP_ITM	3+ strikes in the money	0.70 - 0.85	Expensive, moves like spot, low theta risk

6.2 IV Estimation from VIX

To price options accurately, you need an estimate of Implied Volatility (IV). The system uses a three-tier fallback approach to estimate IV from India VIX, which is a publicly available real-time volatility index published by the NSE. The first tier uses the real-time VIX value directly if the API call succeeds. The second tier uses the most recently cached VIX value if the API call fails but a recent value is available. The third tier falls back to a hardcoded default IV percentage (configured in Config) if neither real-time nor cached data is available. This three-tier approach ensures that the system never crashes due to missing VIX data, while still preferring the most accurate and current data when it is available.

6.3 Spread Modeling (VIX Regimes)

The bid-ask spread on option contracts varies significantly depending on market volatility. In calm markets, spreads are tight and execution is cheap. In volatile markets, spreads widen dramatically and slippage can erode profits. The Options Engine models this reality using three volatility regimes based on VIX levels. Each regime applies a different spread multiplier to the estimated option price, ensuring that the system's cost estimates are realistic across all market conditions.

Regime	VIX Range	Spread Multiplier	Typical Conditions
Low Volatility	< 14	1.0x (tight spreads)	Calm trending markets, low event risk
Normal Volatility	14 - 22	1.5x (moderate spreads)	Typical trading days, moderate event risk
High Volatility	> 22	2.5x (wide spreads)	Budget, elections, global crises, panic

6.4 Position Sizing Formula

Position sizing is the final calculation the Options Engine performs before passing the order to the execution layer. The formula is: $\text{Lots} = (\text{Capital} \times \text{Risk Per Trade}) / (\text{Stop Loss Distance} \times \text{Lot Size})$. For example, if you have Rs 5,00,000 in capital, risk 2% per trade (Rs 10,000), the stop loss distance is 100 points, and the lot size is 25, then $\text{Lots} = 10,000 / (100 \times 25) = 4$ lots. This formula ensures that no single trade can lose more than the configured risk percentage, regardless of how wide or narrow the stop loss is. The result is capped by max_lots from Config to prevent excessively large positions during periods of very tight stops.

7. Module 6 — Black-Scholes (The Calculator)

The Black-Scholes module is the system's theoretical pricing calculator. It implements the famous Black-Scholes option pricing model, which was developed by Fischer Black and Myron Scholes in 1973 and won the Nobel Prize in Economics. While real market prices are set by supply and demand, Black-Scholes gives you a 'fair value' estimate — a baseline that tells you whether an option is expensive or cheap relative to its theoretical worth. The DDLJ system uses this calculator primarily to estimate the cost of an option before placing the order, and to compute the Greeks (risk measures) that inform position management decisions.

7.1 The Greeks — Explained Simply

The Greeks are sensitivity measures that tell you how an option's price is expected to change when different market variables move. They are called 'Greeks' because they are represented by Greek letters. Understanding the Greeks is essential for managing risk in options trading, because each Greek measures a different dimension of risk. Think of them as the dashboard instruments on a car: speedometer tells you how fast you are going, fuel gauge tells you how far you can go, temperature gauge tells you if the engine is overheating. Each Greek is a different gauge for a different risk.

Greek	Measures	Kid-Friendly Explanation	Why It Matters
Delta	Price sensitivity	How much the option moves per 1 rupee move in the index	Tells you how responsive your option is to index moves — higher delta means more bang for your buck
Gamma	Delta's rate of change	How fast Delta itself changes as the index moves	Warns you that Delta is unstable — ATM options have highest gamma and can shift rapidly
Theta	Time decay	How much value the option loses every day	Reminds you that options are wasting assets — every day you hold, you lose some value to time decay
Vega	Volatility sensitivity	How much the option price changes when volatility moves by 1%	Shows how much your option's value depends on market fear — high VIX = high Vega = expensive options

7.2 Why Real VIX Data Matters

The Black-Scholes model requires a volatility input to produce accurate prices. If you feed it a stale or estimated volatility, the output prices will be wrong — sometimes dramatically wrong. This is why the DDLJ system insists on using real-time VIX data whenever possible. India VIX is a market-derived measure of expected volatility over the next 30 days, computed from Nifty option prices by the NSE. It is the closest thing to a 'market consensus' on future volatility, and it updates in real time throughout the trading day. Using VIX as the volatility input to Black-Scholes produces theoretical prices that closely match actual market prices, which in turn produces accurate cost estimates, accurate Greek calculations, and ultimately better trading decisions.

When VIX data is unavailable (for example, during a brief API outage), the system falls back through its three-tier IV estimation process, but it also marks the trade as having 'degraded pricing confidence.' This

means the trader is explicitly warned that the cost estimate may be less accurate than usual. In extreme cases, the system may choose to skip the trade entirely rather than enter with unreliable pricing data. This conservative approach protects against a common failure mode in algorithmic trading: executing trades based on stale or incorrect assumptions.

8. Module 7 — Cost Calculator (The Accountant)

Every trade incurs costs beyond just the option premium. Brokerage fees, government taxes, exchange fees, and stamp duties all eat into your profit — or add to your loss. The Cost Calculator module is the system's accountant: it calculates every single rupee of transaction cost before a trade is placed, so that the Signal Engine's risk-reward calculation accounts for the true net cost of the trade. Ignoring transaction costs is one of the most common mistakes in algorithmic trading. A strategy that looks profitable on paper can become a consistent loser once real costs are factored in. The Cost Calculator ensures this never happens in the DDLJ system.

8.1 Zerodha Transaction Costs Breakdown

The DDLJ system trades through Zerodha, India's largest discount broker. Zerodha's fee structure is well-defined and transparent, which makes it easy to model precisely. The following table lists every cost component that the system accounts for on each options trade. All rates are current as of the document version date and are applied per order side (buy and sell are calculated separately).

Component	Rate / Amount	Charged By	When Applied
Brokerage	Flat Rs 20 per executed order (per side)	Zerodha	Every buy and sell order
STT/CTT	0.0625% on sell side (premium value)	Government of India	Only on the selling leg
Exchange Transaction Charge	0.05% on sell side (premium value)	NSE	Only on the selling leg
GST	18% on (Brokerage + Exchange Charges)	Government of India	Every buy and sell order
SEBI Turnover Fee	Rs 10 per crore of premium	SEBI	Every buy and sell order
Stamp Duty	0.003% on buy side (premium value)	State Government	Only on the buying leg

8.2 Cost Impact on Trading Decisions

Understanding the cost structure is not just an accounting exercise — it directly affects trading decisions. Because brokerage is a flat Rs 20 per side, the effective brokerage percentage decreases as trade size increases. A Rs 5,000 premium trade pays 0.8% in brokerage alone, while a Rs 50,000 premium trade pays only 0.08%. This creates a natural incentive to avoid very small trades where costs dominate. The Cost Calculator enforces this by adding a minimum premium threshold in Config: if the estimated premium is below this threshold, the trade is rejected as 'not economically viable.' Additionally, the total transaction cost (including all components) is subtracted from the Signal Engine's reward estimate before the minimum RR filter is applied. This ensures that the risk-reward ratio always reflects the true net reward after costs, not the gross reward before costs. A trade that appears to have a 2:1 RR before costs might only have a 1.5:1 RR after costs — a significant difference that could mean the difference between a profitable and unprofitable system over hundreds of trades.

9. Module 8 — Data Fetcher (The Pipeline)

The Data Fetcher module is the system's connection to the outside world. It communicates with the Zerodha Kite API to download historical and real-time market data — the OHLCV candles that feed every other module in the system. Without data, the entire engine grinds to a halt. The Data Fetcher handles the complexity of API communication so that no other module needs to know anything about HTTP requests, rate limits, authentication tokens, or data formats. It presents a clean, simple interface: 'Give me the last N candles for instrument X on timeframe Y,' and it handles the rest.

9.1 Auto-Chunking and Caching

The Kite API limits the number of candles that can be retrieved in a single request. For example, you cannot request 300 days of 15-minute candles in one call — the API will reject it. The Data Fetcher handles this transparently through auto-chunking: it automatically splits a large request into multiple smaller API calls, fetches each chunk, and stitches the results together into a single continuous dataset. The calling module never knows that the data was fetched in multiple pieces — it just receives the complete candle series it asked for. Additionally, the Data Fetcher implements a local cache that stores recently fetched data in memory. If the Bias Engine and the Signal Engine both request 60m candles for BankNifty within the same minute, the second request is served from cache instead of making a duplicate API call. This reduces API usage, improves response time, and keeps the system well within rate limits.

9.2 Rate Limiting and Retry Logic

The Kite API enforces rate limits to prevent abuse. If you exceed the limit, the API returns an error instead of data. The Data Fetcher implements two layers of protection against this. First, it tracks the number of API calls made in each time window and proactively delays requests that would exceed the limit. Second, if a rate-limit error does occur despite the proactive throttling, the module implements exponential backoff retry logic: it waits 1 second, then 2, then 4, then 8, up to a maximum of 5 retries. If all retries fail, it returns an error to the calling module, which can decide whether to proceed with stale data or halt the system. This graceful degradation ensures that temporary API issues never cause the system to crash or enter an undefined state.

Interval	Max Candles per Request	Typical Use Case
minute	60	Intraday micro-analysis (rarely used)
3minute	100	Short-term scalping signals
5minute	100	Alternative entry timeframe
15minute	200	Signal Engine entry timing
60minute	200	Bias Engine direction
day	200	Long-term trend confirmation

The Data Fetcher also handles the conversion of raw API responses (JSON format) into the system's internal candle representation — a structured data class with Open, High, Low, Close, Volume, and

Timestamp fields. This conversion happens at the fetch boundary so that every downstream module works with clean, validated data objects instead of raw dictionaries or lists. If any candle fails validation (for example, $\text{High} < \text{Low}$ due to a data error), the module logs a warning and either drops the corrupted candle or interpolates from neighbors, depending on the severity. Data quality is paramount — garbage in, garbage out — and the Data Fetcher serves as the gatekeeper that ensures only clean data enters the pipeline.

10. Module 9 — Token Manager (The Gatekeeper)

The Token Manager is the system's authentication gatekeeper. Every interaction with the Zerodha Kite API requires a valid access token, and managing that token's lifecycle is a surprisingly complex task. Tokens expire, network errors can interrupt the refresh process, and the system must handle all of these edge cases gracefully without ever leaving the trader in a state where the system thinks it is connected but the API has actually rejected the session. The Token Manager encapsulates all of this complexity into a clean interface: other modules simply call 'get_session()' and receive a valid, authenticated Kite client object. They never need to know about request tokens, access tokens, or session validation.

10.1 Token Lifecycle

The token lifecycle in the DDLJ system follows a three-step process that runs every trading day. Understanding this process is important because a failure at any step can prevent the system from trading, and the error messages from Zerodha can be confusing if you do not know what is happening behind the scenes.

Step	Action	Input	Output	Failure Mode
1. Request Token	User logs in to Zerodha via browser and authorizes the app	API Key + Redirect URL	request_token (valid for a few minutes)	User does not complete login; token expires before use
2. Access Token	System exchanges request_token for access_token via API	API Key + API Secret + request_token	access_token (valid for the trading day)	Network error during exchange; invalid request_token
3. Session Validation	System verifies the access_token by fetching user profile	access_token	Validated Kite session object	Token is rejected by API; session is expired or revoked

10.2 Daily Token Refresh

Zerodha access tokens are valid only for the current trading day. This means the Token Manager must perform the full three-step process every morning before the market opens. The system automates this as much as possible: it opens the Zerodha login page, waits for the user to complete authentication, captures the redirect URL containing the request_token, exchanges it for an access_token, and validates the session — all without requiring manual intervention beyond the initial browser login. The access_token is cached in memory for the rest of the day and is used for all subsequent API calls. The Token Manager also implements a heartbeat check every 30 minutes during trading hours: it makes a lightweight API call (such as fetching the user profile) to confirm that the session is still active. If the heartbeat fails, the system triggers an alert and attempts an automatic reconnection. If reconnection fails, the system enters a safe mode where it stops placing new orders but continues monitoring existing positions until the session is restored or the user manually intervenes.

11. Complete Strategy Flow

Now that every module has been explained individually, it is time to see how they work together in a complete trading cycle. This section walks through the entire flow from the moment the system starts up to the moment a trade is closed and recorded. Think of it as watching the entire assembly line in motion — each station does its job and passes the product to the next station. The flow is strictly sequential and unidirectional: data flows from left to right, and no module ever sends data backward. This discipline ensures that there are no circular dependencies, no feedback loops, and no race conditions.

11.1 Step-by-Step Walkthrough

Step 1 — Startup & Authentication

The system starts, the Token Manager obtains a valid session, and the Config module loads all parameters. The Data Fetcher performs a warm-up fetch to populate the initial candle cache for both timeframes.

Step 2 — Bias Calculation

Every new 60m candle triggers the Bias Engine. It computes EMA, ATR, and slope on the 60m series, evaluates the three bias signals, checks for overextension, and publishes the current bias.

Step 3 — Signal Scanning

Every new 15m candle triggers the Signal Engine. It computes indicators on the 15m series, checks for EMA Cross or Continuation patterns, applies all filters, and — if a pattern is detected and bias aligns — publishes an entry signal with SL and target.

Step 4 — Options Translation

The Options Engine receives the signal, selects the appropriate strike, estimates IV from VIX, runs Black-Scholes pricing, applies the spread model, calculates transaction costs, computes position size, and publishes the final order specification.

Step 5 — Order Execution

The order is sent to Zerodha Kite for execution. The system waits for confirmation and records the fill price, timestamp, and order ID. If the order fails (for example, due to insufficient margin), the system logs the error and does not retry automatically — it alerts the user instead.

Step 6 — Position Management

While the position is open, the system continuously monitors the market against the position management rules described below. Each rule is checked on every new 15m candle.

Step 7 — Trade Closure & Recording

When an exit condition is triggered, the system closes the position, calculates the final P&L; (including all transaction costs), and records the complete trade in the journal for later analysis.

11.2 Position Management Rules

The following eight rules govern how open positions are managed. These rules are checked in priority order on every new 15m candle. The first rule that triggers determines the exit action. If no rule triggers, the position remains open and the system waits for the next candle.

#	Rule Name	Condition	Action
1	Stop Loss Hit	Price crosses the SL level	Exit immediately — close the full position
2	Target Hit	Price reaches the target level	Exit immediately — book full profit
3	Near Target	Price comes within 10% of target	Tighten SL to entry (risk-free trade)
4	Breakeven Move	Price moves 1x ATR in favor after Near Target	Move SL to entry + 0.5x ATR
5	Trailing Stop	Price moves 2x ATR in favor from entry	Trail SL to last swing low/high
6	Bias Flip	60m bias changes direction	Exit immediately — the trend has reversed
7	Time Exit	Position is open past 3:15 PM	Close position — avoid overnight risk
8	Force Close	Market approaches closing time (3:25 PM)	Close all open positions — end of day

11.3 Exit Handling Process

When an exit is triggered, the system follows a strict process to ensure the position is closed cleanly and all records are updated. First, it calculates the exact exit price (current market price for stop losses and bias flips; target price for target hits). Second, it sends a closing order to Zerodha and waits for confirmation. Third, it computes the gross P&L; (entry vs exit price), subtracts all transaction costs using the Cost Calculator, and arrives at the net P&L; Fourth, it records the complete trade details — entry time, exit time, entry price, exit price, SL, target, gross P&L;, net P&L;, transaction costs, bias at entry, bias at exit, signal type, option details, and any overrides that were active — in the trade journal. This comprehensive record enables detailed post-trade analysis and continuous strategy improvement. Finally, the system resets its state to 'flat' (no position) and resumes scanning for new signals. The entire exit process takes less than a second in normal conditions.

12. Lego Piece Connections

Every lego piece in the DDLJ system has input ports and output ports. The output of one piece plugs into the input of the next, and the connections are always one-directional. No piece depends on the output of a piece that comes after it in the chain. This section maps out exactly how the pieces connect, what data flows between them, and which pieces depend on which other pieces. Understanding this map is essential for debugging, testing, and extending the system. If a trade goes wrong, you can trace the data flow backward from the trade record to find exactly which piece produced the problematic output.

12.1 Data Flow Diagram (Textual)

The following diagram shows the complete data flow through the system. Each arrow represents a data dependency — the module at the arrow's tail produces data that the module at the arrow's head consumes. Read the diagram from top to bottom, following the arrows, to trace the path from raw market data to a completed trade record.

```
Kite API ===(candles)====> Data Fetcher
Data Fetcher ===(60m OHLCV)====> Indicators (60m)
Data Fetcher ===(15m OHLCV)====> Indicators (15m)
Data Fetcher ===(VIX value)====> Options Engine
Indicators (60m) ===(EMA, ATR, Slope, Swings)====> Bias Engine
Indicators (15m) ===(EMA, ATR, Slope, Swings)====> Signal Engine
Bias Engine ===(BULLISH/BEARISH/NEUTRAL)====> Signal Engine
Signal Engine ===(Entry + SL + Target)====> Options Engine
Options Engine ===(Option order spec)====> Execution Layer
Execution Layer ===(Fill confirmation)====> Position Manager
Position Manager ===(Exit signal)====> Execution Layer
Execution Layer ===(Trade record)====> Trade Journal
Config --- (parameters) ---> [All Modules]
Token Manager --- (session) ---> Data Fetcher, Execution Layer
Cost Calculator --- (transaction costs) ---> Options Engine, Trade Journal
Black-Scholes --- (option price + Greeks) ---> Options Engine
```

12.2 Module Dependency Map

The dependency map shows which modules directly depend on which other modules. A module listed in the 'Depends On' column must be initialized and running before the module in the 'Module' column can function. This map is critical for understanding the startup order and for knowing which modules are affected if a particular module fails or is disabled.

Module	Depends On	Depended On By
Config	Nothing (root module)	All other modules
Token Manager	Config	Data Fetcher, Execution Layer
Data Fetcher	Config, Token Manager	Indicators, Options Engine

Module	Depends On	Depended On By
Indicators	Config, Data Fetcher	Bias Engine, Signal Engine
Bias Engine	Config, Indicators	Signal Engine, Position Manager
Signal Engine	Config, Indicators, Bias Engine	Options Engine
Black-Scholes	Config, Data Fetcher (VIX)	Options Engine
Cost Calculator	Config	Options Engine, Trade Journal
Options Engine	Config, Signal Engine, Data Fetcher, Black-Scholes, Cost Calculator	Execution Layer
Execution Layer	Config, Options Engine, Token Manager	Position Manager, Trade Journal
Position Manager	Config, Bias Engine, Execution Layer	Execution Layer, Trade Journal

Notice that Config and Token Manager are at the bottom of the dependency tree — they have no dependencies of their own, and everything else depends on them. This is why the startup sequence always initializes Config first, then Token Manager, then Data Fetcher, then Indicators, and so on up the chain. If any module in the chain fails to initialize, the system halts and reports the failure rather than starting in a partially functional state. It is better to have no system than a broken system that makes unpredictable decisions.

13. Key Design Decisions

Every trading system is the product of hundreds of design decisions, each one reflecting a trade-off between competing priorities. This section documents the most important design decisions in the DDLJ system and explains the reasoning behind each one. Understanding why the system is designed the way it is will help you make informed decisions when you need to customize or extend it. Blindly changing a parameter without understanding its purpose can easily break the system's logic and produce unexpected results.

13.1 Why Two Timeframes?

Using a single timeframe for both direction and timing creates a fundamental conflict: the timeframe that is sensitive enough to catch entry signals is also too noisy to reliably determine trend direction. Conversely, the timeframe that is smooth enough to show the true trend is too slow to provide timely entries. The solution is to separate the two concerns onto two different timeframes. The 60m chart is slow enough to filter out intraday noise and show the real trend, while the 15m chart is fast enough to catch meaningful entry patterns. This dual-timeframe approach is one of the oldest and most validated concepts in technical analysis, used by traders from Paul Tudor Jones to Linda Raschke. The key insight is that the two timeframes serve fundamentally different purposes and should never be mixed — bias comes from the higher timeframe, timing from the lower timeframe, and never the other way around.

13.2 Why Three-Signal Bias?

A bias engine that relies on a single indicator is fragile. If that indicator fails — and every indicator fails sometimes — the system will trade in the wrong direction with full confidence. Using three independent signals provides redundancy: if one signal misfires, the other two can overrule it. This is the same principle behind triple modular redundancy in aerospace engineering, where critical systems have three independent computers that vote on every decision. The DDLJ system uses the same approach: EMA Position is the primary voter, and EMA Slope and Market Structure are the backup voters. The system only trades when at least two out of three signals agree, and it requires EMA Position to be one of the agreeing signals. This ensures that the most important indicator always has veto power over the final decision.

13.3 Why ATR-Based Stops?

Fixed-point stop losses (e.g., 'always place the stop 50 points away') fail because they do not adapt to changing market conditions. In a quiet market, 50 points might be an enormous move that never gets hit, meaning your stop is so far away that you risk too much. In a volatile market, 50 points might be a normal intraday fluctuation, meaning your stop gets hit constantly and you are whipsawed out of good trades. ATR-based stops solve this by scaling the stop distance to the market's own volatility. When the market is quiet, ATR is small and stops are tight. When the market is volatile, ATR is large and stops are wide. The stop is always '2x ATR' — but what that means in absolute points changes every day based on what the market is actually doing. This adaptive approach has been shown in extensive backtesting to produce significantly better results than fixed stops across a wide range of market conditions.

13.4 Why ITM Options?

OTM (Out-of-The-Money) options are tempting because they are cheap, but they have three critical flaws for a trend-following system. First, they have low delta (typically 0.20-0.40), which means they barely move even when the index moves significantly. You might be right about the direction and still lose money because the option did not respond enough. Second, they have high theta decay — they lose value rapidly as expiration approaches, even if the index does not move against you. Third, they often have wider bid-ask spreads, making them more expensive to enter and exit. ITM options, by contrast, have higher delta (0.55-0.70), lower theta, and tighter spreads. They behave more like a leveraged version of the underlying index, which is exactly what a trend-following system wants: a vehicle that amplifies the index's directional move without the time-decay headwind that OTM options face. The slightly higher premium is a small price to pay for significantly better risk-adjusted returns.

13.5 Why Real VIX Data?

The Black-Scholes model's accuracy depends entirely on the quality of its volatility input. Feed it a rough estimate, and you get a rough price — potentially off by 20-30% or more. Feed it real-time VIX, and you get a price that is typically within 2-5% of the actual market price. This difference matters enormously when you are calculating position sizes, estimating transaction costs, and deciding whether a trade is economically viable. A 20% error in option price estimation could mean the difference between a trade that meets your minimum RR threshold and one that does not. Using real VIX data also ensures that the system's cost estimates are accurate across different market regimes — calm, normal, and volatile — because VIX itself reflects the current regime. A hardcoded IV estimate would be wrong most of the time and would require constant manual adjustment. Real VIX data makes the system self-calibrating.

This concludes the Strategy Core Documentation for DDLJ Trading System v9.1.0. Every module has been described, every connection has been mapped, and every key decision has been explained. You now have a complete understanding of how the system works — from the Config module's control panel to the Trade Journal's after-action report. Remember: the system is a set of lego pieces. Each piece is simple. The complexity comes from how they connect. Master the pieces, and you master the engine.